

Digital Nurture 5.0

Python Full Stack Engineer Track

HANDS-ON EXERCISE BOOK

Database Integration: PostgreSQL, MySQL & MongoDB

Week 4–5 | 4 Days | 7 Hands-On Exercises

How to Use This Exercise Book

This exercise book covers Module 3 — Database Integration for the Digital Nurture 5.0 Deep Skilling Program. All 7 hands-on exercises are built around a single, realistic scenario (a Student Course Registration System) so your work builds cohesively from one exercise to the next.

Software Requirements

- PostgreSQL (latest) — <https://www.postgresql.org/download/>
- MySQL Community Server 8.x — <https://dev.mysql.com/downloads/mysql/>
- MongoDB Community Server — <https://www.mongodb.com/try/download/community>
- MongoDB Compass (GUI) — <https://www.mongodb.com/products/tools/compass>
- Python 3.10+ — <https://www.python.org/downloads/>
- VS Code — <https://code.visualstudio.com/download>
- pip packages: psycopg2-binary, mysql-connector-python, pymongo, sqlalchemy, flask-sqlalchemy

Submission Guidelines

- Organise your solutions in a folder named `Module3_DatabaseIntegration/<YourName>/`
- For SQL exercises: save each exercise as a `.sql` file (e.g., `hands_on_1.sql`)
- For Python/ORM exercises: save as `.py` files inside a subfolder (e.g., `orm/`)
- Upload the folder to your GitHub repository and share the URL with your POC

NOTE	Complete Hands-On 1 through 4 before attempting 5, 6, and 7. Each exercise builds on the schema and data created in earlier ones.
-------------	---

Difficulty Guide

Level	Hands-On	What to Expect
Beginner	1, 2	Core SQL, schema design, basic CRUD
Intermediate	3, 4, 5	Advanced queries, MongoDB, optimisation
Advanced	6, 7	ORM integration, migrations, Python backend

Common Scenario: Student Course Registration System

All exercises in this book use the following scenario. Read it carefully — every hands-on task refers to this system.

A college wants to digitise its Course Registration process. You are the backend developer tasked with designing the database, writing queries, optimising performance, integrating MongoDB for unstructured data, and finally wiring everything up to a Python backend using an ORM.

Database Schema — Relational (PostgreSQL / MySQL)

Create a database named `college_db`. Use the following tables throughout all exercises.

1. students

Column Name	Data Type	Description / Constraint
<code>student_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT
<code>first_name</code>	VARCHAR(50)	NOT NULL
<code>last_name</code>	VARCHAR(50)	NOT NULL
<code>email</code>	VARCHAR(100)	UNIQUE, NOT NULL
<code>date_of_birth</code>	DATE	
<code>department_id</code>	INT	FOREIGN KEY → departments
<code>enrollment_year</code>	INT	

2. departments

Column Name	Data Type	Description / Constraint
<code>department_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT
<code>dept_name</code>	VARCHAR(100)	NOT NULL
<code>hod_name</code>	VARCHAR(100)	Head of Department
<code>budget</code>	DECIMAL(12, 2)	

3. courses

Column Name	Data Type	Description / Constraint
<code>course_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT
<code>course_name</code>	VARCHAR(150)	NOT NULL
<code>course_code</code>	VARCHAR(20)	UNIQUE
<code>credits</code>	INT	
<code>department_id</code>	INT	FOREIGN KEY → departments

4. enrollments

Column Name	Data Type	Description / Constraint
<code>enrollment_id</code>	INT	PRIMARY KEY, AUTO_INCREMENT
<code>student_id</code>	INT	FOREIGN KEY → students

Column Name	Data Type	Description / Constraint
course_id	INT	FOREIGN KEY → courses
enrollment_date	DATE	
grade	CHAR(2)	A, B, C, D, F — nullable

5. professors

Column Name	Data Type	Description / Constraint
professor_id	INT	PRIMARY KEY, AUTO_INCREMENT
prof_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE
department_id	INT	FOREIGN KEY → departments
salary	DECIMAL(10,2)	

Sample Data (use for all exercises)

Run the following INSERT statements once to populate your database. This data is referenced in all hands-on exercises.

```
-- departments
INSERT INTO departments (dept_name, hod_name, budget) VALUES
('Computer Science', 'Dr. Ramesh Kumar', 850000.00),
('Electronics', 'Dr. Priya Nair', 620000.00),
('Mechanical', 'Dr. Suresh Iyer', 540000.00),
('Civil', 'Dr. Ananya Sharma', 430000.00);

-- students
INSERT INTO students (first_name, last_name, email, date_of_birth, department_id,
enrollment_year) VALUES
('Arjun', 'Mehta', 'arjun.mehta@college.edu', '2003-04-12', 1, 2022),
('Priya', 'Suresh', 'priya.suresh@college.edu', '2003-07-25', 1, 2022),
('Rohan', 'Verma', 'rohan.verma@college.edu', '2002-11-08', 2, 2021),
('Sneha', 'Patel', 'sneha.patel@college.edu', '2004-01-30', 3, 2023),
('Vikram', 'Das', 'vikram.das@college.edu', '2003-09-14', 1, 2022),
('Kavya', 'Menon', 'kavya.menon@college.edu', '2002-05-17', 2, 2021),
('Aditya', 'Singh', 'aditya.singh@college.edu', '2004-03-22', 4, 2023),
('Deepika', 'Rao', 'deepika.rao@college.edu', '2003-08-09', 1, 2022);

-- courses
INSERT INTO courses (course_name, course_code, credits, department_id) VALUES
('Data Structures & Algorithms', 'CS101', 4, 1),
('Database Management Systems', 'CS102', 3, 1),
('Object Oriented Programming', 'CS103', 4, 1),
('Circuit Theory', 'EC101', 3, 2),
('Thermodynamics', 'ME101', 3, 3);

-- enrollments
INSERT INTO enrollments (student_id, course_id, enrollment_date, grade) VALUES
(1, 1, '2022-07-01', 'A'), (1, 2, '2022-07-01', 'B'),
(2, 1, '2022-07-01', 'B'), (2, 3, '2022-07-01', 'A'),
(3, 4, '2021-07-01', 'A'), (4, 5, '2023-07-01', NULL),
(5, 1, '2022-07-01', 'C'), (5, 2, '2022-07-01', 'A'),
(6, 4, '2021-07-01', 'B'), (7, 5, '2023-07-01', NULL),
(8, 1, '2022-07-01', 'A'), (8, 3, '2022-07-01', 'B');
```

```
-- professors
INSERT INTO professors (prof_name, email, department_id, salary) VALUES
  ('Dr. Anand Krishnan', 'anand.k@college.edu', 1, 95000.00),
  ('Dr. Meena Pillai', 'meena.p@college.edu', 1, 88000.00),
  ('Dr. Sunil Rajan', 'sunil.r@college.edu', 2, 82000.00),
  ('Dr. Latha Gopal', 'latha.g@college.edu', 3, 79000.00),
  ('Dr. Kartik Bose', 'kartik.b@college.edu', 4, 76000.00);
```

HANDS-ON 1 [Beginner]**Schema Design & Core SQL — DDL and Normalisation****Topics Covered:**

✓ Database Schema Design	✓ Normalisation (1NF–3NF)
✓ ER Relationships	✓ DDL — CREATE / ALTER / DROP
✓ Referential Integrity	

In this hands-on, you will create the college_db schema from scratch, verify normalisation, and understand how tables relate to each other.

DB	Use PostgreSQL or MySQL for this exercise. Commands shown work in both unless noted otherwise.
-----------	--

Exercise Background

Before writing a single INSERT, a good engineer designs the schema. You will analyse the requirements, draw relationships, and create all tables with proper constraints.

Task 1: Create the Database and Tables

Goal: Practise DDL by creating the full college_db schema with constraints.

Steps:

1. Create a new database named college_db.
2. Write CREATE TABLE statements for all five tables listed in the schema section: departments, students, courses, enrollments, professors.
3. Add NOT NULL, UNIQUE, and PRIMARY KEY constraints where specified in the schema.
4. Define FOREIGN KEY constraints to enforce referential integrity between students → departments, courses → departments, enrollments → students, enrollments → courses, professors → departments.
5. Run the CREATE TABLE statements in your SQL client and verify they execute without errors.

Hint:

- Create departments first — other tables reference it via FOREIGN KEY.
- In MySQL use AUTO_INCREMENT; in PostgreSQL use SERIAL or IDENTITY.

Expected Outcome: All 5 tables created with no errors. DESCRIBE / \d commands show all columns and constraints.

Task 2: Verify Normalisation

Goal: Confirm the schema satisfies 1NF, 2NF, and 3NF — and document why.

Steps:

6. For 1NF: Confirm every column holds atomic values. Identify any hypothetical violation if, for example, we had stored multiple phone numbers in one field.
7. For 2NF: Confirm every non-key column is fully dependent on the entire primary key (especially check the enrollments table, which has a composite candidate key of student_id + course_id).
8. For 3NF: Confirm there are no transitive dependencies. (Hint: would storing dept_name in the students table violate 3NF?)
9. Write a short comment using SQL -- comments inside your .sql file documenting your 3NF analysis for the enrollments table.

Hint:

- A transitive dependency exists when column C depends on column B, which depends on the primary key A, rather than directly on A.

Expected Outcome: Your .sql file contains at least 6 lines of analysis comments explaining 1NF, 2NF, 3NF compliance.

| Task 3: Alter and Extend the Schema

Goal: Use ALTER TABLE to safely extend an existing schema without data loss.

Steps:

10. Add a column phone_number VARCHAR(15) to the students table.
11. Add a column max_seats INT DEFAULT 60 to the courses table.
12. Add a CHECK constraint to enrollments ensuring grade is one of ('A','B','C','D','F') or NULL.
13. Rename the hod_name column in departments to head_of_dept (PostgreSQL: ALTER COLUMN ... RENAME TO; MySQL: ALTER TABLE ... CHANGE ...).
14. Drop the phone_number column you added in step 1 (simulate a schema rollback).

Hint:

- In MySQL 8+, CHECK constraints are enforced. In older versions they are parsed but ignored — always verify.
- Use INFORMATION_SCHEMA.COLUMNS to confirm changes.

Expected Outcome: Each ALTER runs without error. Final schema matches the original plus max_seats and the renamed head_of_dept column.

HANDS-ON 2 [Beginner]**Writing SQL Queries — DML, Joins & Aggregations****Topics Covered:**

✓ DML — INSERT / UPDATE / DELETE	✓ SELECT with WHERE / ORDER BY
✓ INNER JOIN, LEFT JOIN, multi-table JOINS	✓ Aggregate Functions — COUNT, AVG, SUM, MAX
✓ GROUP BY and HAVING	

Now that the schema is ready, you will write DML statements and progressively complex SELECT queries against the college_db data.

| Task 1: Insert, Update and Delete Data

Goal: Practise all DML operations ensuring data consistency.

Steps:

15. Insert the sample data from the Common Schema section into all five tables (use the provided INSERT statements).
16. Insert two additional students of your own choosing into the students table.
17. Update the grade of student_id = 5 for course_id = 1 from 'C' to 'B'.
18. Delete enrollments where grade IS NULL (students who never received a grade).
19. Verify row counts using SELECT COUNT(*) after each operation.

Hint:

- Run SELECT * FROM enrollments WHERE grade IS NULL before deleting to preview the rows that will be removed.

Expected Outcome: students table has 10 rows. enrollments has only rows with non-NULL grades after the DELETE.

| Task 2: Single-Table Queries and Filtering

Goal: Build confidence with SELECT, WHERE, ORDER BY, and LIKE.

Steps:

20. Retrieve all students enrolled in 2022, ordered by last_name alphabetically.
21. Find all courses with more than 3 credits, sorted by credits descending.
22. List all professors whose salary is between 80,000 and 95,000.
23. Find all students whose email ends with '@college.edu' using the LIKE operator.
24. Count the total number of students per enrollment_year.

Hint:

- BETWEEN is inclusive on both ends.
- The % wildcard matches any sequence of characters in LIKE patterns.

Expected Outcome: Query 5 returns 3 rows — one per distinct enrollment_year in the sample data.

| Task 3: Multi-Table Joins

Goal: Write JOIN queries spanning 2–4 tables to answer real business questions.

Steps:

25. List each student's full name (first_name + ' ' + last_name) alongside the name of their department. (JOIN students and departments.)
26. Show each enrollment along with the student's name and the course name. (3-table JOIN: enrollments, students, courses.)

27. Find all students who are NOT enrolled in any course using a LEFT JOIN and WHERE ... IS NULL pattern.
28. Display every course along with the number of students enrolled in it. Courses with zero enrolments must still appear. (LEFT JOIN courses with enrollments, GROUP BY course.)
29. List each department along with its professors and their salaries. Include departments that have no professors yet.

Hint:

- LEFT JOIN keeps all rows from the left table even if no match exists on the right — essential for 'find missing relationships' queries.

Expected Outcome: Query 3 returns students with no enrollments. Query 4 shows all 5 courses, including those with 0 students.

| Task 4: Aggregations and Grouping

Goal: Use aggregate functions to generate summary reports.

Steps:

30. Calculate the total number of enrollments per course. Display course_name and enrollment_count.
31. Find the average salary of professors per department. Round to 2 decimal places.
32. Find all departments where the total budget exceeds 600,000.
33. Show the grade distribution for course CS101: count of each grade (A, B, C, D, F).
34. Using HAVING, list departments where more than 2 students are enrolled across all courses in that department.

Hint:

- Use GROUP BY with HAVING (not WHERE) to filter on aggregated results.
- ROUND(AVG(salary), 2) works in both PostgreSQL and MySQL.

Expected Outcome: Query 2 returns 4 rows — one per department, with average salary rounded to 2 decimal places.

HANDS-ON 3 [Intermediate]**Advanced SQL — Subqueries, Views & Transactions****Topics Covered:**

✓ Subqueries (correlated & non-correlated)	✓ Views — creation, usage, updatable views
✓ Stored Procedures (MySQL) / Functions (PostgreSQL)	✓ Transactions — COMMIT, ROLLBACK, SAVEPOINT
✓ Indexes and Query Plans	

You will now write advanced SQL including subqueries, create views for reusability, use stored procedures/functions, and manage data integrity with transactions.

| Task 1: Subqueries

Goal: Use subqueries to answer questions that require multi-step logic.

Steps:

35. Find all students who are enrolled in more courses than the average number of enrollments per student. (Use a non-correlated subquery to calculate the average.)
36. List courses in which all enrolled students have received a grade of 'A'. (Correlated subquery or NOT EXISTS.)
37. Find the professor with the highest salary in each department using a correlated subquery.
38. Using a subquery in the FROM clause (derived table), calculate the per-department average salary and then filter to departments where that average exceeds 85,000.

Hint:

- A correlated subquery re-executes for every row of the outer query — use it only when a non-correlated version is not possible, as it can be slow.
- Derived tables in the FROM clause must always have an alias.

Expected Outcome: Query 1 should return students enrolled in 3 or more courses based on sample data.

| Task 2: Creating and Using Views

Goal: Create reusable views that simplify complex queries.

Steps:

39. Create a view vw_student_enrollment_summary showing each student's full name, department, number of courses enrolled in, and GPA (average grade converted: A=4, B=3, C=2, D=1, F=0).
40. Create a view vw_course_stats showing course_name, course_code, total_enrollments, and avg_gpa for each course.
41. Query vw_student_enrollment_summary to find students with GPA above 3.0.
42. Attempt to UPDATE a row through vw_student_enrollment_summary and note what happens. Research and document in your comments why multi-table views are generally not updatable.
43. DROP both views and recreate vw_student_enrollment_summary as a view WITH CHECK OPTION (use a single-table subset view for this step).

Hint:

- CASE WHEN grade='A' THEN 4 WHEN grade='B' THEN 3 ... END converts letter grades to GPA points.
- WITH CHECK OPTION prevents INSERT/UPDATE through the view if the row would not be visible through the view's WHERE clause.

Expected Outcome: SELECT * FROM vw_course_stats returns 5 rows — one per course in the sample data.

| Task 3: Stored Procedures and Transactions

Goal: Encapsulate business logic in procedures and use transactions to maintain integrity.

Steps:

44. Write a stored procedure `sp_enroll_student` (MySQL) or function `fn_enroll_student` (PostgreSQL) that accepts `student_id`, `course_id`, and `enrollment_date`, checks for duplicate enrollment, and inserts the record.
45. Write a procedure `sp_transfer_student` that moves a student from one department to another. Wrap the `UPDATE` and a log-insert into a new table `department_transfer_log` inside a single transaction. `ROLLBACK` if either statement fails.
46. Test the transaction by manually introducing an error (e.g., invalid foreign key) and verify that the first `UPDATE` is also rolled back.
47. Use `SAVEPOINT` to create a mid-transaction checkpoint: insert two enrollment records; set a `SAVEPOINT` after the first; deliberately fail the second; `ROLLBACK TO SAVEPOINT` and verify only the first record was saved.

Hint:

- In MySQL use `DELIMITER $$ ... DELIMITER ;` to define stored procedures in a `.sql` file.
- In PostgreSQL use `CREATE OR REPLACE FUNCTION ... LANGUAGE plpgsql`.

Expected Outcome: `sp_enroll_student` raises a descriptive error on duplicate enrollment. The `SAVEPOINT` test shows the first insert persisting after partial rollback.

HANDS-ON 4 [Intermediate]**Query Optimisation — Indexes, EXPLAIN & the N+1 Problem****Topics Covered:**

✓ Index Types — B-Tree, Composite, Partial	✓ EXPLAIN / EXPLAIN ANALYZE
✓ Query Plans — Seq Scan vs Index Scan	✓ N+1 Query Problem
✓ Connection Pooling (concept)	

Performance is critical in production databases. In this exercise, you will measure query performance, add indexes, read query plans, and identify the notorious N+1 problem.

| Task 1: Baseline Performance — No Indexes

Goal: Establish a baseline before any optimisation.

Steps:

48. Run EXPLAIN (PostgreSQL) or EXPLAIN FORMAT=JSON (MySQL) on the following query and save the output as a comment in your .sql file: `SELECT s.first_name, s.last_name, c.course_name FROM enrollments e JOIN students s ON s.student_id = e.student_id JOIN courses c ON c.course_id = e.course_id WHERE s.enrollment_year = 2022;`
49. Identify whether the query plan shows a Sequential Scan (Postgres) or Full Table Scan (MySQL) on any table.
50. Note the estimated cost (PostgreSQL) or rows examined (MySQL) in your comments.

Hint:

- EXPLAIN does not execute the query — it only shows the plan. EXPLAIN ANALYZE (PostgreSQL) actually runs it and shows real timings.
- A Seq Scan is not always bad for small tables — it becomes a problem as data grows.

Expected Outcome: Your comment block contains the EXPLAIN output and identifies at least one table scan.

| Task 2: Add Indexes and Compare Plans

Goal: Create indexes and observe their impact on query plans.

Steps:

51. Create a B-Tree index on `students.enrollment_year`.
52. Create a composite UNIQUE index on `enrollments(student_id, course_id)` — this also prevents duplicate enrollments.
53. Create an index on `courses.course_code`.
54. Re-run the EXPLAIN from Task 1 and compare the new plan to the baseline. Document the change (Seq Scan → Index Scan?) as a comment.
55. Create a partial index on `enrollments(student_id) WHERE grade IS NULL` to optimise lookups for unevaluated enrollments.

Hint:

- Composite index column order matters — put equality filter columns first, range filter columns last.
- Partial indexes (PostgreSQL) only index rows matching a condition — smaller and faster for specific query patterns.

Expected Outcome: EXPLAIN output shows an Index Scan on at least one table. The composite UNIQUE index prevents duplicate enrollment inserts.

| Task 3: Identify and Fix the N+1 Problem

Goal: Recognise and resolve the most common ORM performance anti-pattern.

Steps:

56. Simulate the N+1 problem in Python: fetch all enrollments with `SELECT * FROM enrollments`, then loop through each row and issue a separate `SELECT` to fetch the student's name. Count the total queries executed.
57. Rewrite the script using a single `JOIN` query that retrieves all enrollment records with student names in one query.
58. Compare the number of database round-trips between the two approaches and log the difference using Python's `time` module.
59. Document in comments: in a real application with 10,000 enrollments, how many extra queries would the N+1 version issue?

Hint:

- N+1 means: 1 query to get N rows, then N additional queries to fetch related data — N+1 total instead of 1.
- This is most commonly introduced by ORM lazy loading. The fix is eager loading (`JOIN` or ORM `select_related` / `joinedload`).

Expected Outcome: Script version 1 prints '13 queries executed'. Script version 2 prints '1 query executed'. Both return identical data.

HANDS-ON 5 [Intermediate]**MongoDB — Document Modelling, CRUD & Aggregation****Topics Covered:**

✓ Documents & Collections	✓ BSON Types
✓ CRUD Operations	✓ Aggregation Pipeline
✓ Indexes in MongoDB	✓ Embedding vs Referencing

MongoDB stores data as flexible JSON-like documents rather than rigid rows. In this exercise, you will model part of the college system in MongoDB, practise all CRUD operations, and build an aggregation pipeline.

TOOL	Use MongoDB Compass for a visual experience, or the mongosh shell. Both are acceptable.
-------------	---

MongoDB Schema Design for the College System

In MongoDB, you will model Course Feedback data — information that varies per course and student (star ratings, text comments, tags) and is not well-suited to a fixed relational schema.

```
// feedback collection – sample document
{
  _id: ObjectId(),
  student_id: 1,           // references students table (relational DB)
  course_code: 'CS101',    // references courses table
  semester: '2022-ODD',
  rating: 4,               // integer 1-5
  comments: 'Excellent teaching. Would recommend.',
  tags: ['challenging', 'well-structured', 'good-examples'],
  submitted_at: ISODate('2022-11-30T10:15:00Z'),
  attachments: [
    { filename: 'notes.pdf', size_kb: 240 }
  ]
}
```

Task 1: Create the Collection and Insert Documents

Goal: Set up the MongoDB collection and insert realistic feedback documents.

Steps:

60. In MongoDB Compass or mongosh, create a database named college_nosql.
61. Create a collection named feedback.
62. Insert at least 10 feedback documents following the schema above. Vary ratings (1–5), tags, and semesters. Include at least 3 documents for CS101 and 2 for CS102.
63. Insert one document that intentionally omits the attachments field — MongoDB's schema-less design allows this.
64. Verify the inserts using `db.feedback.countDocuments()`.

Hint:

- In mongosh: use `college_nosql` then `db.feedback.insertMany([...])`.
- `ObjectId` is auto-generated if you omit `_id`.

Expected Outcome: `db.feedback.countDocuments()` returns 10 or more. One document has no attachments field.

Task 2: CRUD Operations

Goal: Practise all four CRUD operations on the feedback collection.

Steps:

65. READ: Find all feedback documents where rating is 5.

66. READ: Find feedback for course CS101 where the tags array contains 'challenging'. Use \$elemMatch or a simple array value query.
67. READ: Retrieve only the student_id, course_code, and rating fields (projection) for all documents — exclude _id.
68. UPDATE: For all feedback documents with rating < 3, add a field needs_review: true using updateMany and \$set.
69. UPDATE: Push a new tag 'reviewed' into the tags array of all documents where needs_review is true, using \$push.
70. DELETE: Delete all feedback documents where the semester is '2021-EVEN'.

Hint:

- Projection syntax: db.feedback.find({}, { student_id:1, course_code:1, rating:1, _id:0 }).
- \$elemMatch is needed when querying sub-documents with multiple conditions; for a simple value match in an array, use the value directly.

Expected Outcome: READ query 2 returns only CS101 feedback with the 'challenging' tag. UPDATE adds needs_review field to low-rated docs.

| Task 3: Aggregation Pipeline

Goal: Build a multi-stage aggregation pipeline to generate analytical reports.

Steps:

71. Write a pipeline that: (Stage 1) filters to semester '2022-ODD'; (Stage 2) groups by course_code calculating average rating and total feedback count; (Stage 3) sorts by average rating descending.
72. Extend the pipeline with a \$project stage to rename avg_rating to average_rating and round it to 1 decimal place using \$round.
73. Write a pipeline that uses \$unwind on the tags array, then \$group by tag to count how many times each tag appears. Sort by count descending — a tag frequency leaderboard.
74. Add an index on course_code and verify its usage with db.feedback.find({course_code:'CS101'}).explain('executionStats') — confirm the stage shows IXSCAN not COLLSCAN.

Hint:

- \$unwind deconstructs an array field — each array element becomes a separate document in the pipeline.
- \$round: ['\$avg_rating', 1] rounds to 1 decimal place.

Expected Outcome: Pipeline 1 returns one document per course with average_rating rounded to 1 decimal. Pipeline 3 shows tag frequency with 'challenging' likely appearing most.

HANDS-ON 6 [Advanced]**ORM Integration — SQLAlchemy & Django ORM****Topics Covered:**

✓ SQLAlchemy Core & ORM	✓ Django ORM (models, queries)
✓ Defining Models and Relationships	✓ CRUD via ORM
✓ Sessions and Connection Pooling	✓ Avoiding N+1 with joinedload / select_related

ORMs let you interact with the database using Python objects instead of raw SQL. In this exercise, you will build a Python module that reproduces key operations from Hands-On 1–4 using SQLAlchemy, and optionally explore the same with Django ORM.

INSTALL	<code>pip install sqlalchemy psycopg2-binary flask-sqlalchemy</code> (add <code>mysql-connector-python</code> if using MySQL)
----------------	---

Task 1: SQLAlchemy — Define Models and Connect

Goal: Set up the ORM models that mirror the college_db schema.

Steps:

75. Create a Python file `models.py`. Import necessary classes from `sqlalchemy`.
76. Define an engine connecting to your college_db PostgreSQL (or MySQL) database.
77. Define five ORM model classes: `Department`, `Student`, `Course`, `Enrollment`, `Professor` — mapping to the same columns as your SQL schema.
78. Define relationships: `Student` has a many-to-one relationship to `Department`. `Enrollment` has many-to-one relationships to both `Student` and `Course`.
79. Use `Base.metadata.create_all(engine)` to auto-create tables in a fresh database (`college_db_orm`), and confirm they appear in your SQL client.

Hint:

- `from sqlalchemy import Column, Integer, String, ForeignKey, Date, Numeric`
- `from sqlalchemy.orm import relationship, declarative_base`
- Use `relationship('ClassName', back_populates='...')` for bidirectional navigation.

Expected Outcome: Running `python models.py` creates all 5 tables in `college_db_orm` with correct columns and constraints visible in your SQL client.

Task 2: CRUD Operations via ORM

Goal: Perform all CRUD operations using SQLAlchemy's session API.

Steps:

80. Create a `crud.py` file. Open a Session using `sessionmaker`.
81. INSERT: Add 3 `Department` objects and 5 `Student` objects. Commit the session.
82. INSERT: Add 3 `Course` objects and 4 `Enrollment` objects linking students to courses.
83. READ: Query all students in department 'Computer Science' using `session.query(Student).join(Department).filter(Department.dept_name == 'Computer Science')`.
84. READ: Query all enrollments and print each student's name alongside course name. Enable `echo=True` on the engine to count SQL queries issued.
85. UPDATE: Find a specific student by email and update their `enrollment_year`. Commit.
86. DELETE: Remove an enrollment record using `session.delete(enrollment_obj)`. Commit and verify.

Hint:

- Always commit or rollback after mutations. `session.commit()` flushes pending changes to the DB.

- Enable SQLAlchemy query logging with `create_engine(..., echo=True)` to print all SQL — invaluable for detecting N+1.

Expected Outcome: With `echo=True`, READ task 5 may reveal N+1. Document how many queries are issued and proceed to Task 3 to fix it.

| Task 3: Eager Loading to Fix N+1

Goal: Use `joinedload` to eliminate N+1 queries in ORM code.

Steps:

87. Identify the N+1 query from Task 2 Step 5 by counting the SQL log lines with `echo=True`.
88. Rewrite the query using `joinedload`:

```
from sqlalchemy.orm import joinedload
session.query(Enrollment).options(joinedload(Enrollment.student), joinedload(Enrollment.course)).all()
```
89. Count the SQL queries again with `echo=True` — it should now be 1 query (or 3 using the subquery loading strategy).
90. Compare the two outputs and document the difference in a comment block at the top of `crud.py`.
91. (Bonus) Achieve the same with Django ORM's `select_related`:

```
Enrollment.objects.select_related('student', 'course').all()
```

Hint:

- `joinedload` issues one SQL with JOINS. `subqueryload` issues one query per relationship using IN subqueries. Both eliminate N+1.

Expected Outcome: With `joinedload`, all enrollment + student + course data loads in 1 SQL statement. Comment block confirms query count went from 13 to 1.

HANDS-ON 7 [Advanced]**Migrations & Versioning — Alembic and Django Migrations****Topics Covered:**

✓ Migration Concepts	✓ Alembic for SQLAlchemy
✓ Django Migrations	✓ Migration History and Version Control
✓ Rollback Strategies	

Migrations allow your database schema to evolve alongside your code. In this final exercise, you will use Alembic (SQLAlchemy) or Django's built-in migration system to version-control schema changes and practise safe rollbacks.

INSTALL	<code>pip install alembic</code> (Alembic is the migration tool for SQLAlchemy; Django ships with migrations built-in)
----------------	--

| Task 1: Set Up Alembic and Create a Baseline Migration

Goal: Initialise Alembic and generate the first migration from your existing models.

Steps:

92. In your project folder, run: `alembic init migrations`
93. Edit `alembic.ini` to set `sqlalchemy.url` to your `college_db_orm` connection string.
94. Edit `migrations/env.py` to import your models' Base and set `target_metadata = Base.metadata`.
95. Run `alembic revision --autogenerate -m 'initial schema'` to generate the first migration file.
96. Inspect the generated file in `migrations/versions/` — confirm it contains the `upgrade()` and `downgrade()` functions.
97. Run `alembic upgrade head` to apply the migration. Verify the `alembic_version` table is created in your database.

Hint:

- `--autogenerate` compares your models to the current database state and generates the diff automatically.
- `alembic_version` tracks which migration version is currently applied.

Expected Outcome: `alembic current` shows the revision hash. The `alembic_version` table exists in `college_db_orm`. All 5 tables are present.

| Task 2: Add and Apply Incremental Migrations

Goal: Simulate real-world schema evolution with multiple migration revisions.

Steps:

98. In `models.py`, add a new column `is_active` `BOOLEAN DEFAULT TRUE` to the Student model.
99. Run `alembic revision --autogenerate -m 'add is_active to students'`.
100. Inspect the generated migration — confirm `upgrade()` adds the column and `downgrade()` drops it.
101. Apply with `alembic upgrade head`. Verify the column exists using your SQL client.
102. Add a new table `CourseSchedule` with columns: `schedule_id`, `course_id` (FK), `day_of_week`, `start_time`, `end_time`. Create the model, generate the migration, and apply it.
103. Run `alembic history --verbose` to see the full migration chain.

Hint:

- Always inspect autogenerated migrations before applying — Alembic may miss some changes (e.g., column renames) and you may need to edit the file manually.

Expected Outcome: `alembic history` shows 3 revisions. The `students` table has `is_active` column. The `course_schedules` table exists.

| Task 3: Rollback and Recovery

Goal: Practise safely reverting to a previous schema version.

Steps:

104. Note the current alembic head revision hash using alembic current.
105. Roll back one step using alembic downgrade -1. Confirm the is_active column is dropped.
106. Roll back to the very first revision using alembic downgrade base (this undoes all migrations).
107. Re-apply all migrations with alembic upgrade head and confirm you are back to the latest state.
108. Bonus — Django Migrations: If using Django, run python manage.py makemigrations and python manage.py migrate. Then demonstrate rollback with python manage.py migrate <app_name> <previous_migration_number>.

Hint:

- alembic downgrade -1 steps back one revision. alembic downgrade base removes all applied migrations.
- Always back up production databases before running downgrade in a real project.

Expected Outcome: After downgrade -1, the is_active column is gone. After upgrade head, it is back. alembic current matches the expected head hash.